Finishline Infra Project Assignment

Terraform Modular Infrastructure on AWS (us-east-1)

Reporter: Joseph Ndzoh Dong

Start date: Thursday February 26th 2026

End date: Tuesday March 2nd, 2026

Team Lead: Ferdinand

Assignee: Finish Line Seniors

Objective

Build a production-style AWS foundation using Terraform modules. Provision networking, a shared ALB pattern for Kubernetes ingresses, a secure jumphost, an EKS cluster, and remote Terraform state in S3. Deliver a repeatable, documented project suitable for handoff.

Scenario

Your team is standing up the “Finishline” platform in AWS. You must provide a network baseline, an access entrypoint (jumphost), an EKS cluster, and a shared Application Load Balancer intended to be used by all Kubernetes ingresses.

Architecture

Diagram 1: AWS resources and traffic flow

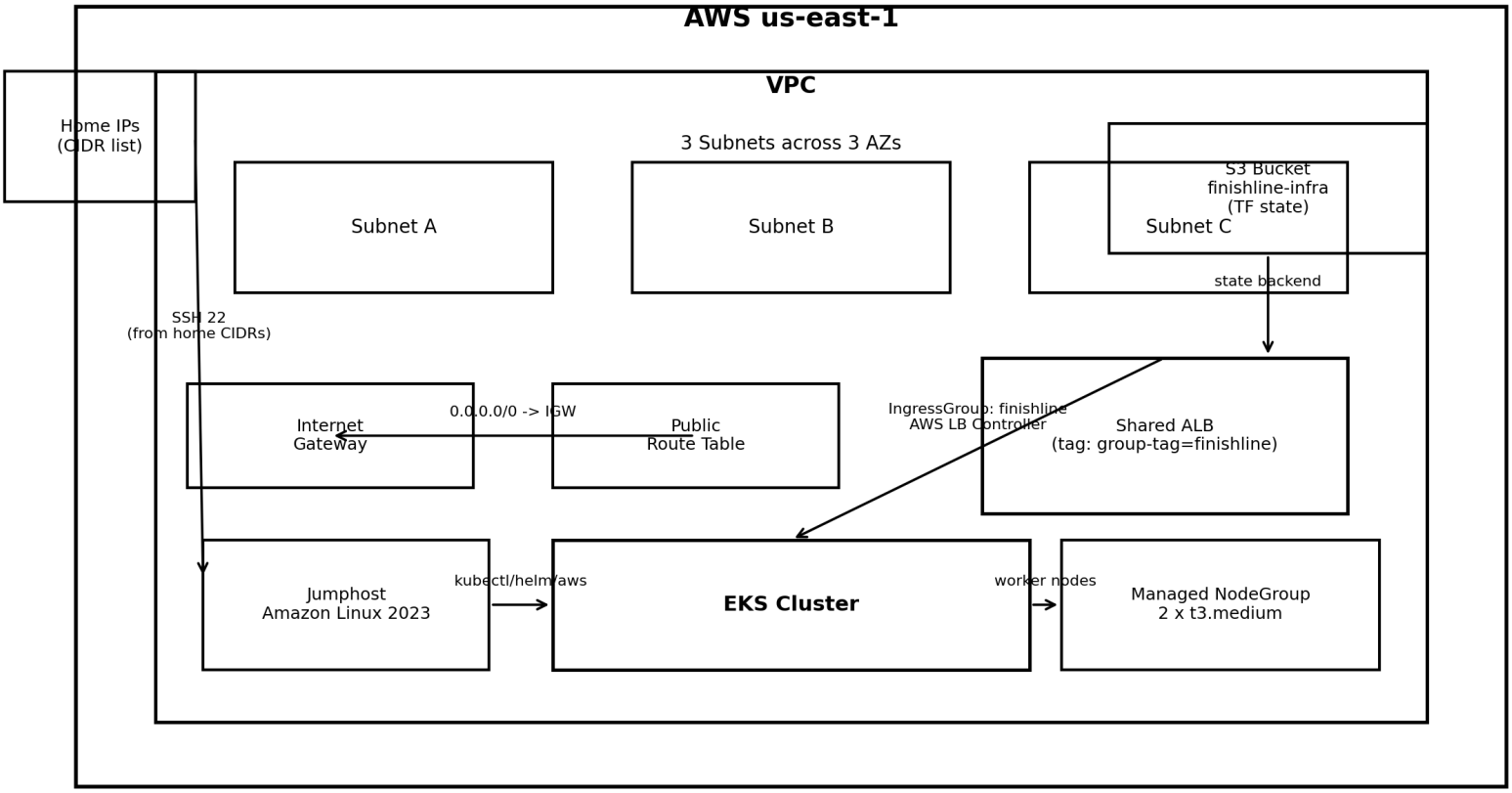
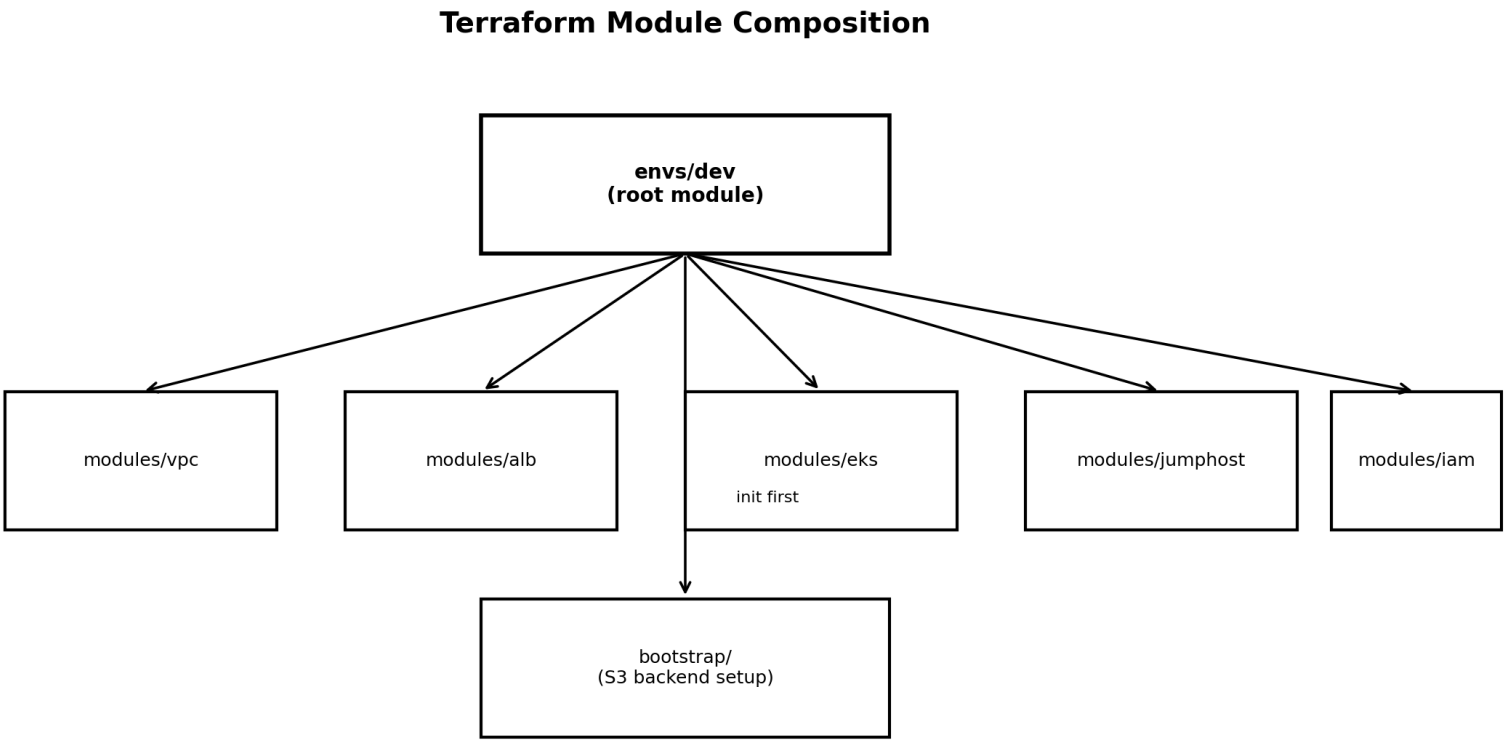


Diagram 2: Terraform module composition



Requirements

A) VPC with 3 subnets and networking

Provision a VPC in us-east-1 with exactly 3 subnets across 3 Availability Zones. Include all required networking constructs to support an internet-facing ALB, a public jumphost, and EKS nodes.

Must include:

- CIDR plan documented in README
- Three subnets across three AZs
- Internet Gateway
- Route tables and associations
- Security groups / NACL decisions documented
- Outputs for VPC and subnet IDs

B) Single ALB shared by all Kubernetes ingresses

Provision a single Application Load Balancer with tag group-tag=finishline. Document show how all Kubernetes ingresses will share this ALB using the AWS Load Balancer Controller IngressGroup mechanism.

Must include:

- ALB is internet-facing unless a private design is justified
- Tag on ALB: group-tag=finishline
- Security group rules for HTTP/HTTPS
- Outputs: ALB ARN and DNS name
- Documentation of required Ingress annotations for shared ALB group usage

C) Jumphost (Amazon Linux 2023) restricted to home IPs

Create a jumphost EC2 instance running Amazon Linux 2023. It must be reachable by SSH only from a variable list of home IP CIDRs. Terraform must generate/manage an SSH keypair and ensure the private key is downloaded locally for the operator.

Must include:

- Security group allows SSH (22) only from home CIDR list
- SSH keypair creation and local key material output/download
- Outputs: jumphost public IP/DNS and ssh command

D) EKS cluster + managed node group (2 x t3.medium)

Provision an EKS cluster in us-east-1 with a managed node group of exactly 2 nodes using t3.medium instances. Node must be using bottlerocket x86 architecture as preferred AMI

Must include:

- EKS cluster and managed node group
- Node group size fixed to 2 (min=2, desired=2, max=2) for this assignment
- Outputs: cluster name/endpoint and node group name
- Validation steps: kubectl get nodes shows 2 Ready nodes

E) Jumphost authenticates to EKS

Ensure the jumphost can authenticate to EKS and run kubectl commands. Implement IAM and EKS access mapping so the instance role can be granted appropriate Kubernetes RBAC permissions. Document the chosen method.

Must include:

- Instance IAM role and least privilege permissions

- EKS access mapping (and explanation)
- Validation: aws eks update-kubeconfig and kubectl get nodes from jumphost

F) Install required tools on the jumphost

On the jumphost install: mysql-client, kubectl, aws-cli v2, latest helm, and latest kustomize. Provide a repeatable method (user-data or configuration management) and a verification checklist.

Verification checklist:

- aws --version (must be v2)
- kubectl version --client
- helm version
- kustomize version
- mysql --version (or equivalent package verification)

G) Remote Terraform state in S3 (finishline-infra)

Create an S3 bucket named finishline-infra and configure Terraform to store state remotely in that bucket. Include a bootstrap workflow that creates the bucket before backend initialization. State locking via DynamoDB is recommended document your decision.

Must include:

- Bootstrap step that creates finishline-infra
- Backend configuration instructions (S3 key naming convention)
- Migration steps from local state to remote state

Required Deliverables

- Terraform repository with modular structure (modules/ + envs/).
- Readme with step-by-step runbook: bootstrap → init → apply → validate.
- Diagrams included in repo (source + rendered images).
- Evidence of successful validation (command outputs or screenshots) in a power point documentation.

Rubric (100 points)

Requirement	Points	Notes (what we look for)
A) VPC (3 subnets + networking) correctness & outputs	20	Meets requirement; evidence included secure defaults.

B) Shared ALB provisioned, tagged (group-tag=finishline), and documented shared-ingress pattern	15	Meets requirement; evidence included secure defaults.
C) Jumphost AML 2023 hardened access (home CIDRs only) + key downloaded locally	15	Meets requirement; evidence included secure defaults.
D) EKS cluster + managed node group (2 x t3.medium) + validation steps	20	Meets requirement; evidence included secure defaults.
E) Jumphost → EKS authentication (IAM + RBAC mapping) + validation proof	10	Meets requirement; evidence included secure defaults.
F) Tooling installed on jumphost + verification evidence	10	Meets requirement; evidence included secure defaults.
G) Remote state in S3 (finishline-infra) + bootstrap workflow (DynamoDB locking bonus)	5	Meets requirement; evidence included secure defaults.
Project quality: module design, readability, tagging, documentation clarity	5	Meets requirement; evidence included secure defaults.

Submission Instructions

Submit a Git repository containing Terraform code, README, and diagrams. Include validation outputs/screenshots. Include a short-Known Issues / Assumptions section.